

- `def fifa_winner(country = "England"):`
- `print("I want " + country + " to win the next FIFA cup")`

In this case, England is the default value but another name can be used to print a different message. When calling the function, if the argument is left empty, it will return the default value. So calling the function,

- `fifa_winner("Croatia")`
- `fifa_winner()`

Will return

- I want Croatia to win the next FIFA cup
- I want England to win the next FIFA cup

You can also use the code to calculate numbers. E.g. if you want the function to run any calculations

- `def product_two_numbers(a, b):`
- `return a * b`

If you call this function and pass the input parameters into the function, it will return the product of the numbers

- `product_two_numbers(3, 5)`

will return the value 15. Or if you use the print command instead of the return command, it will display the values in the display frame. The return statement is used to exit a function and go back to the place from where it was called. The statement can contain an expression which gets evaluated and the value is returned. If there is no expression in the statement or the return statement itself is not present inside a function, then the function will return the *None* object. Let's dig a bit deeper into calling functions.

Arguments

There are 4 types of arguments that can be used to call a function: Required, Keyword, Default and Variable-length arguments. Required arguments, as the name suggests, are where an argument is required as per the of the function. In an earlier example, the function `hello_name` needs an argument to be called.

- `def hello_name(fname):`
- `print("Hello " + fname)`

So when calling this function, if you do not enter an argument in parentheses following the function name, it would not work and will give you a syntax error. If you simply type in `hello_name()` to call this function, you will see the following syntax error: